



BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

- Performance Measure (P) – Measures how successfully the agent performs.
- Environment (E) – The world in which the agent operates.
- Actuators (A) – The mechanisms used by the agent to perform actions.
- Sensors (S) – The mechanisms used by the agent to perceive the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

- Grid dimensions – Define the size of the environment.
- Walls – Represent obstacles in the environment.
- Food positions – Represent locations of food.
- Opponent positions (`self.opponents`) – Represent other agents/entities in the environment.
- Agent position (`self.agent_pos`) – Represents the agent's current location.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent.

The environment is multi-agent because the code contains `self.opponents`, meaning there are other agents/opponents in the environment that can affect the agent's situation or decisions.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything an agent has perceived through its sensors from the beginning of its operation up to the current time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

- `get_percept()` represents the Sensors (S) component of PEAS.
- It collects information about the environment and provides that information to the agent so that it can make decisions.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

- The environment is Partially Observable if `get_percept()` returns only limited information about the environment.
- The agent cannot see every aspect of the environment, such as hidden information about traps, opponents, or other states that are not included in the returned dictionary.
- Therefore, the agent must make decisions using only the information available through its sensors.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

- The Performance Measure (P) is being updated.
- The agent's score changes when it hits a wall, so the performance measure is affected.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

- The performance measure should evaluate the agent's results and behavior in the environment, not how much internal processing it performs.
- For example, hitting a wall affects the external environment interaction and should reduce the score. Measuring processing effort would not necessarily show whether the agent is behaving successfully.
- Therefore, a good performance measure should focus on the actual outcome of the agent's actions.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable because the agent cannot detect the locations of the toxic traps. It must make decisions without complete information about the environment.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. Implementation Instructions: Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The `smells_toxin` sensor gives the agent information about whether it is on a toxic trap. This helps the agent avoid losing points and choose actions that maximize its expected score, making its behaviour more rational.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. Implementation Instructions: In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit was when the performance measure rewarded the agent for turning the vacuum on rather than for keeping the environment clean. The agent could exploit this faulty metric instead of achieving the real goal of cleaning the environment.

Finalize and Lock Lab 01 Analytical Evaluation

